

实验报告

实验 5 存储器及数据通路设计

课程名称	数字逻辑与计算机组成	指导教师	吴海军
------	------------	------	-----

1 实验目的

- 掌握利用 ROM 组件实现指令存储器的设计方法。
- 掌握 ASCII 点阵字库的使用方法。
- 掌握利用 RAM 组件实现按字节存取的数据存储器的设计方法。
- 掌握实现 RV32I 存取指令的设计方法。
- 掌握 RV32I 下地址逻辑部件的设计方法。
- 掌握 RV32I 立即数扩展器的设计方式。
- 根据 RV32I 指令执行过程，掌握 RV32I 运算类指令数据通路的设计方法。
- 掌握跳转控制器的设计方法。

2 实验环境

- 平台：Logisim。

3 实验内容

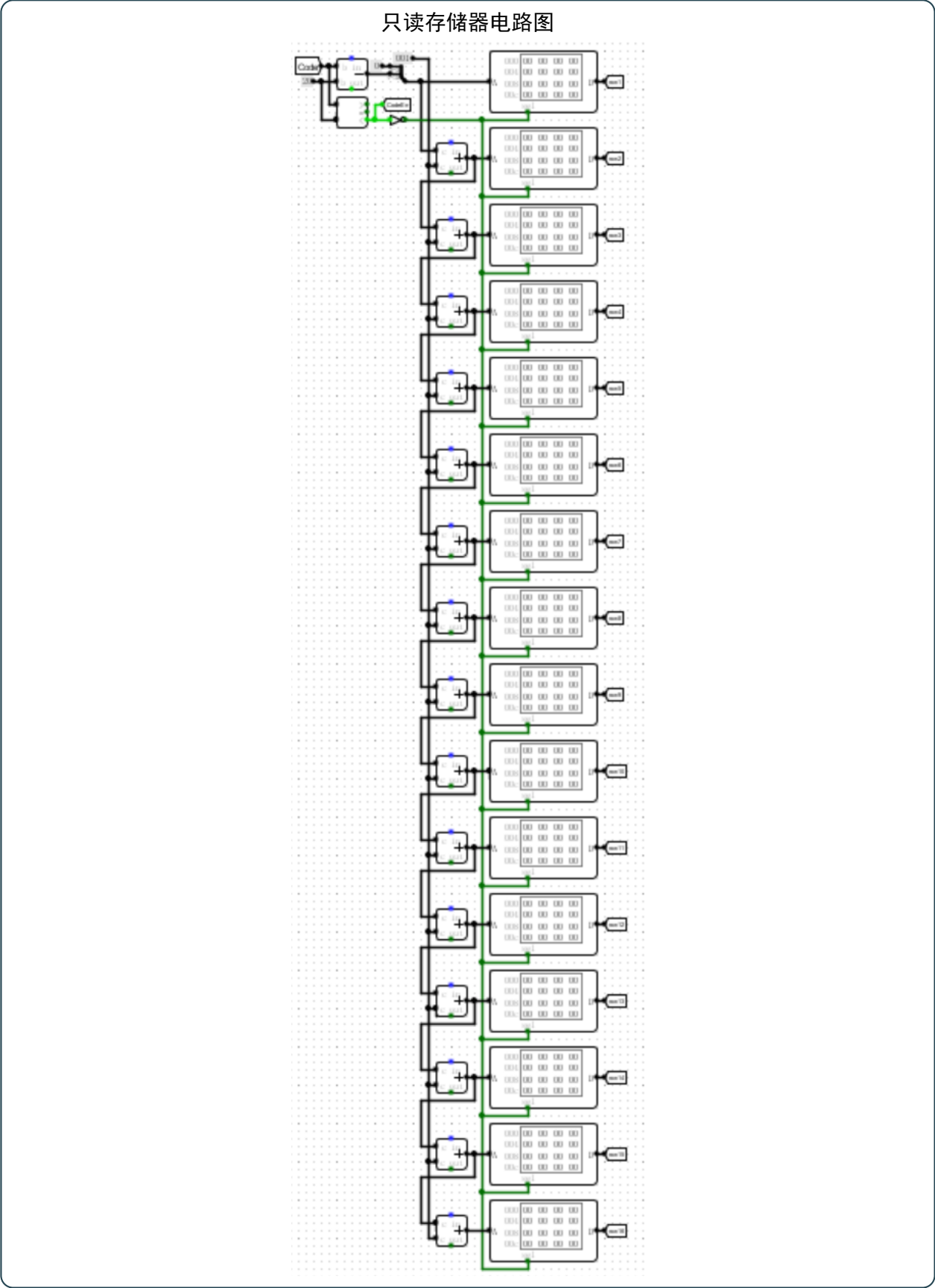
3.1 只读存储器实验

3.1.1 实验整体方案设计

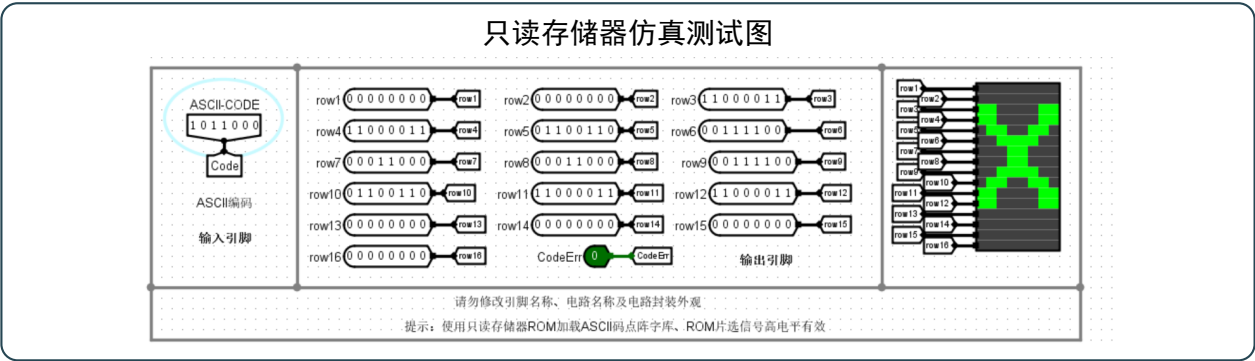
- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：

ASCII-CODE	输入显示字符的 ASCII 码
row1 row15	输出信号
CodeErr	若 code 不在可见字符范围，则 CodeErr 为 1，不显示字符

3.1.2 实验原理图和电路图



3.1.3 实验数据仿真测试图



3.1.4 错误现象及分析

本次实验中没有出现错误。

3.2 数据存储器实验

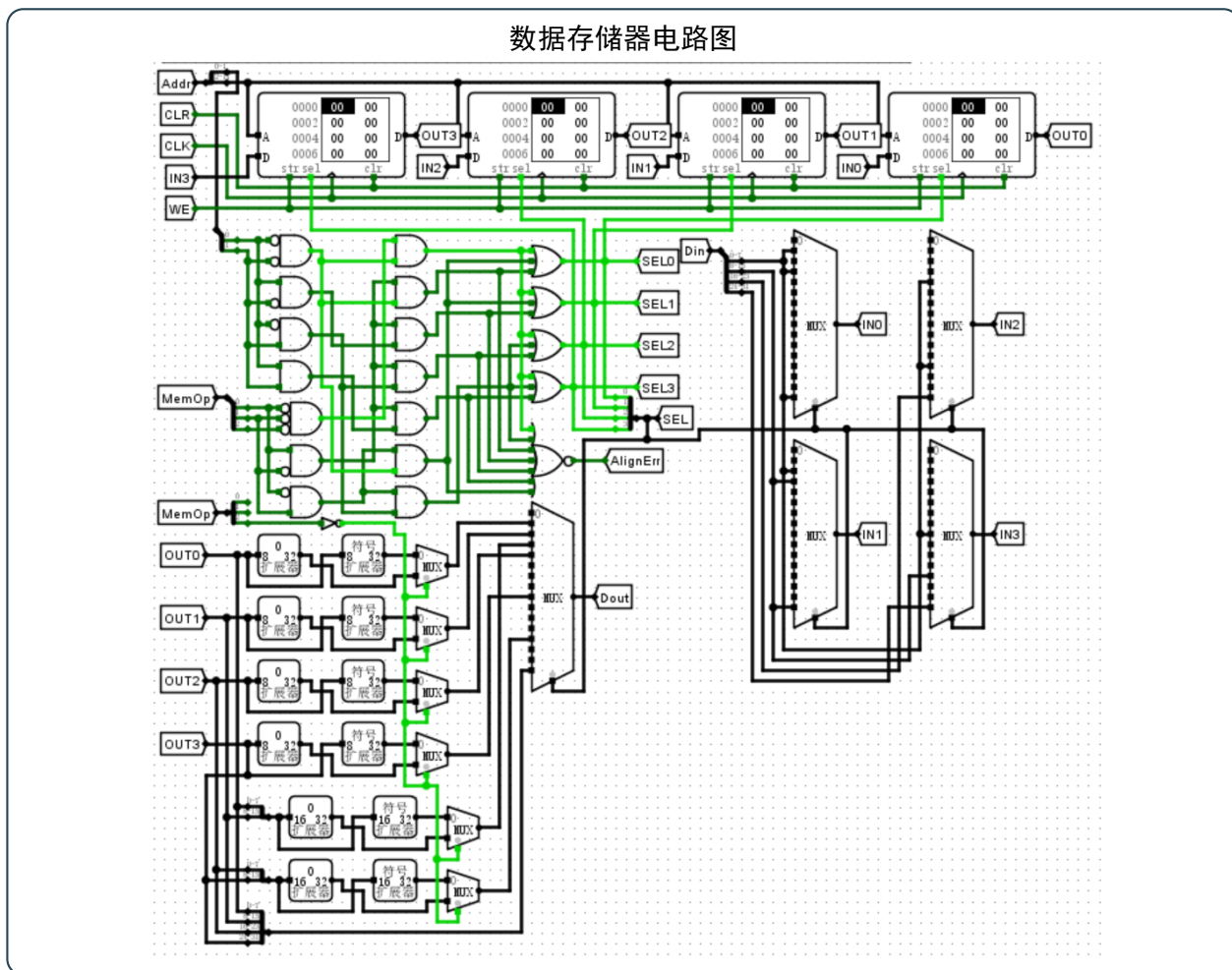
3.2.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：见下第一张表。
- 关键控制信号与地址对齐关系：见下第二张表。

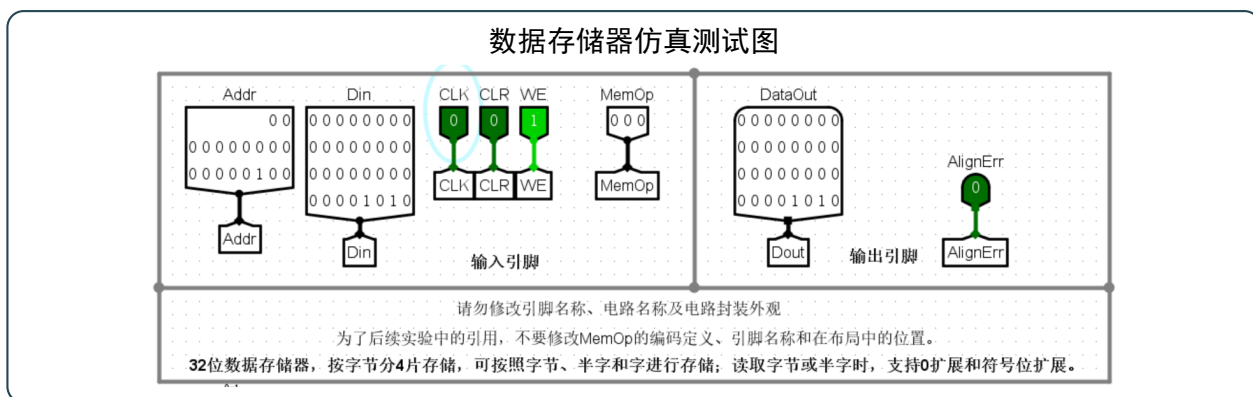
Addr	地址
CLK	时钟信号
WE	写使能信号
CLR	清零信号
Din	输入数据
MemOp	控制信号
Dout	输出数据
AlignErr	错误信号（地址未对齐时为 1）

MemOp	指令	含义
000	lw,sw	按字存取，4 字节
001	lbu	按字节读取，1 个字节，0 扩展到 4 字节
010	lhu	按半字读取，2 字节，0 扩展到 4 字节
101	lb, sb	按字节存取，1 个字节，在读取时，按符号位扩展到 4 字节
110	lh, sh	按半字存取，2 字节，在读取时，按符号位扩展到 4 字节

3.2.2 实验原理图和电路图



3.2.3 实验数据仿真测试图



3.2.4 错误现象及分析

实验中没有出现错误。

3.3 取指令部件 IFU 实验

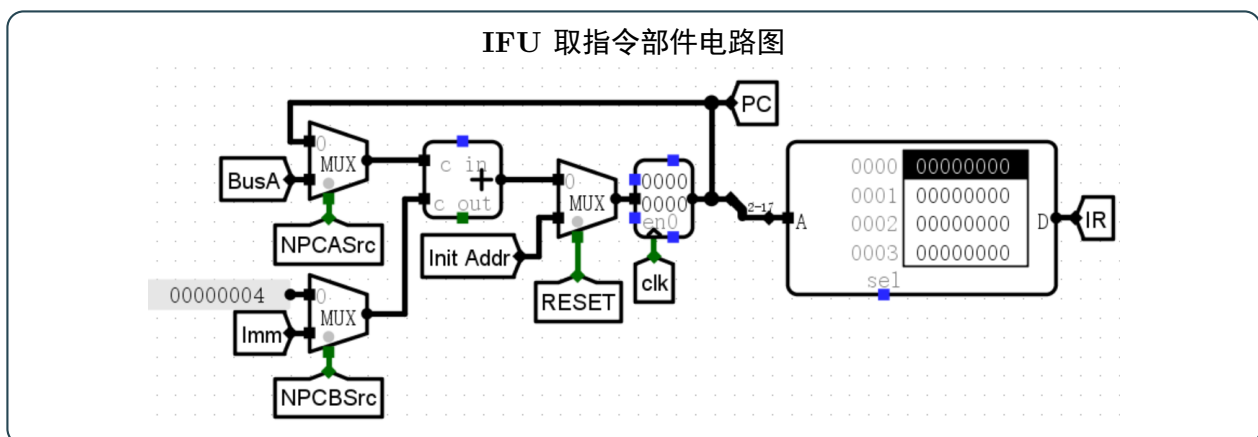
3.3.1 实验整体方案设计

- 顶层模块设计：实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：见下第一张表。
- 关键控制信号：见下第二张表。

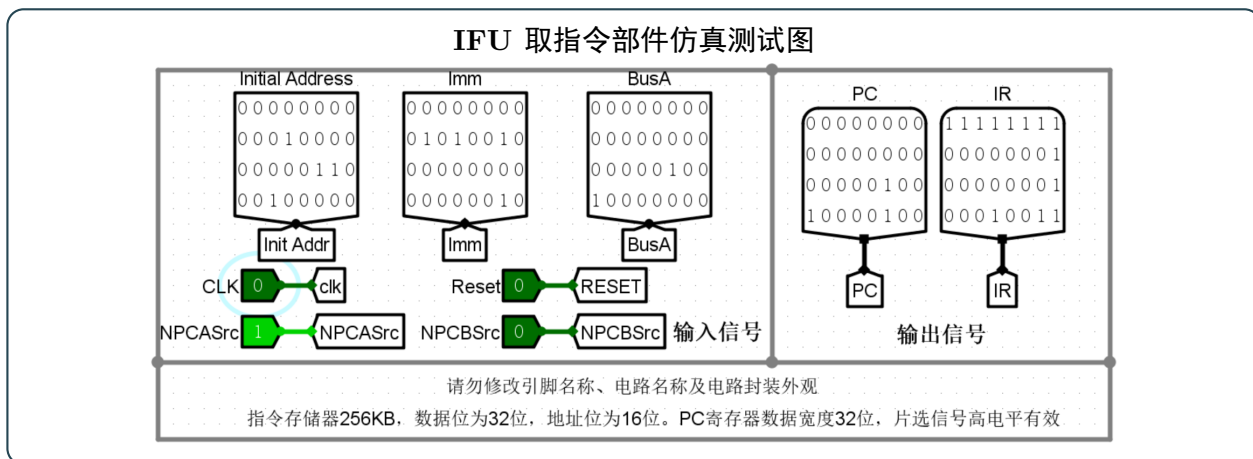
PC	程序计数器
Initial Address	初始地址
Reset	复位初始地址
Imm	程序跳转的偏移量（立即数）
BusA	寄存器 Rs1 中的数据
CLK	时钟信号
NPCASrc	控制 PC 加法器输入端 A 的信号
NPCBSrc	控制 PC 加法器输入端 B 的信号

指令类型	NPCASrc	NPCBSrc	下条指令地址
顺序指令	0	0	$PC=PC+4$
无条件跳转 jal	0	1	$PC=PC+Imm$
无条件跳转 jalr	1	1	$PC=R[rs1]+Imm$
分支跳转指令	0	1	条件成立 $PC=PC+Imm$ ，否则 $PC=PC+4$

3.3.2 实验原理图和电路图



3.3.3 实验数据仿真测试图



	输入						输出（边沿信号有效前）	
序号	Reset	Initial Address	Imm	BusA	NPCASrc	NPCBSrc	PC	IR
1	1	0x400	0	0	0	0	0	0
2	0	0	0	0	0	0	0x400	0x2083
3	0	0	0	0	0	0	0x404	0x8133
4	0	0	0x7f8	0	0	1	0x408	0x106213
5	0	0	0x20c	0	0	1	0xc00	0x413
6	1	0x1500	0	0	0	0	0xe0c	0xdeadbeef
7	0	0	0	0	0	0	0x1500	0x413
8	0	0	0	0	0	0	0x1504	0x9117
9	0	0	0	0x3fbc0	1	1	0x1508	0xffc10113
10	0	0	0	0	0	0	0x3fbc0	0x413
11	0	0	0x1fc	0	0	1	0x3fbc4	0x9117
12	0	0	0	0x150c	1	1	0x3fdc0	0x40513
13	0	0	0x0c00	0	1	1	0x150c	0x39c000ef
14	0	0	0	0	0	0	0xc00	0x413
15	0	0	0	0	0	0	0xc04	0x9117
16	0	0	0xffff9c8	0	0	1	0xc08	0xffc10113
17							0x5d0	0xdeadbeef

3.3.4 错误现象及分析

本实验中没有出现错误。

3.4 取操作数部件 IDU 实验

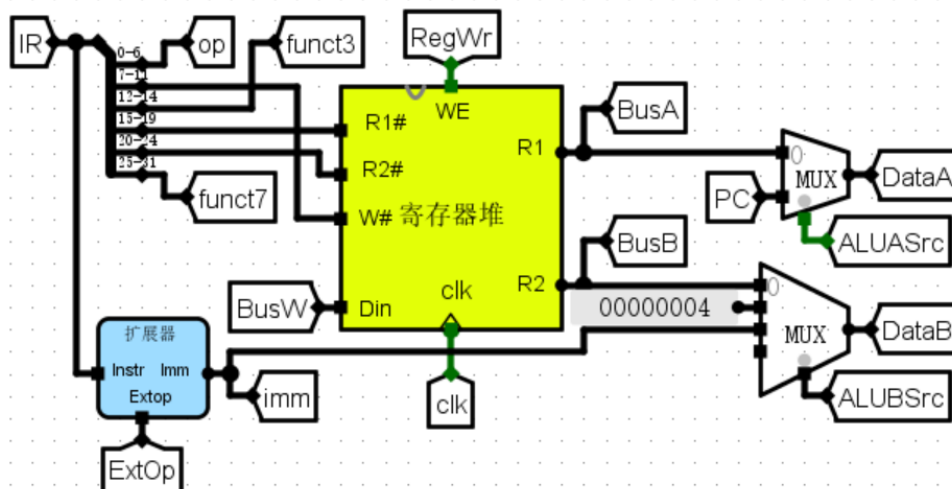
3.4.1 实验整体方案设计

- 顶层模块设计：本实验电路较为简单，不需要顶层模块设计图。
- 输入输出引脚：见下第一张表。

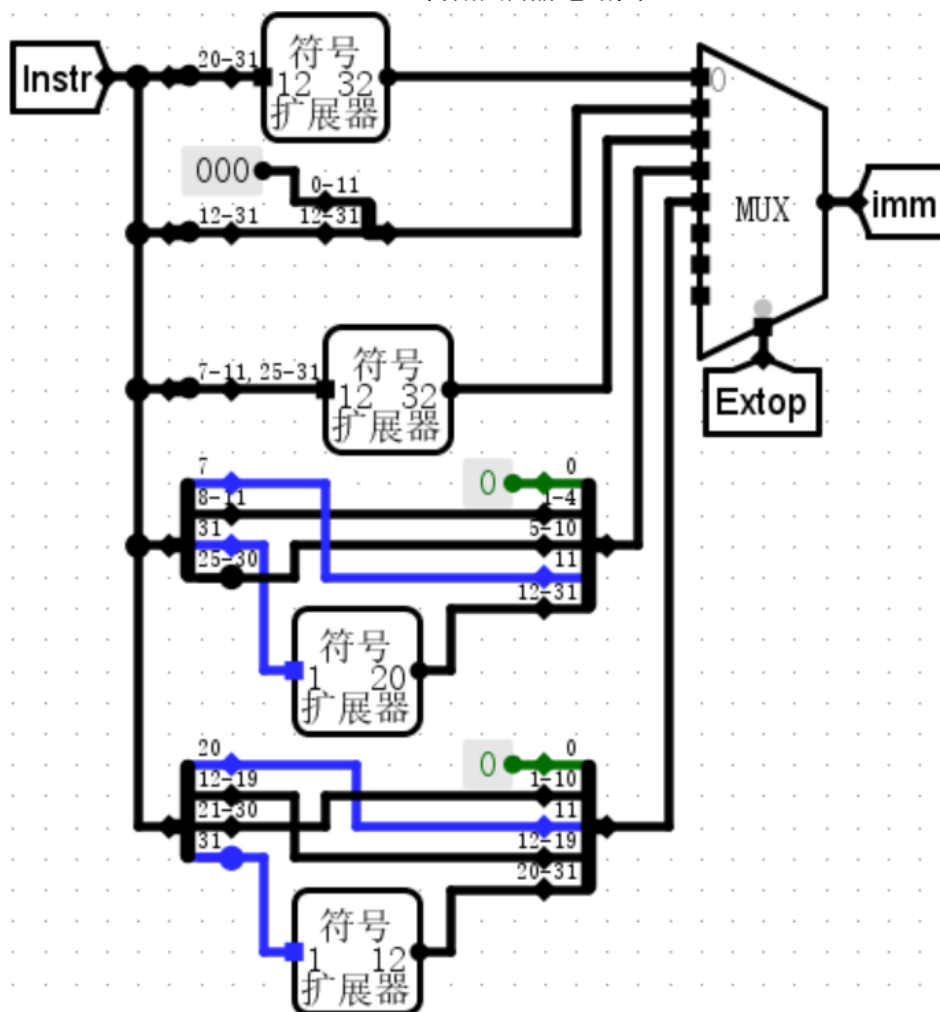
IR	指令
BusW	写入寄存器堆的数据
PC	程序计数器
ExtOp	立即数扩展器控制信号
RegWr	寄存器堆写使能控制信号
ALUASrc	ALU 输入端 A 的来源
ALUBSrc	ALU 输入端 B 的来源
CLK	时钟信号
DataA	ALU 输入端 A 的来源
DataB	ALU 输入端 B 的来源
BusA	寄存器 Rs1 中的数据
BusB	寄存器 Rs2 中的数据
Imm	立即数
OpCode	操作码
Func3,Func7	功能码

3.4.2 实验原理图和电路图

IDU 取操作数部件电路图

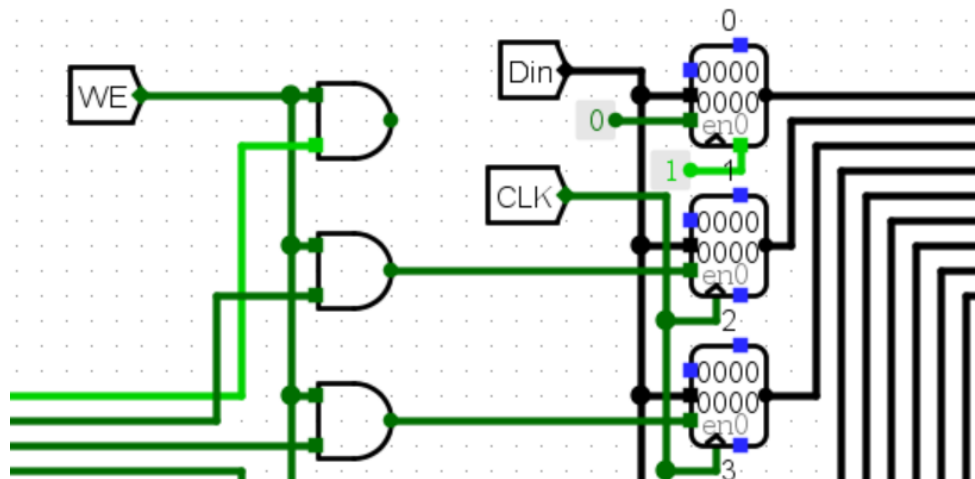


IDU 立即数扩展器电路图



以下为 IDU 寄存器堆在本实验中做出的修改部分，使得第 0 号寄存器始终为 0，且在写入时，如果写入地址为 0，则不进行写入操作。其余均与先前实验中设计的寄存器堆相同。

IDU 寄存器堆修改部分电路图



3.4.3 实验数据仿真测试图

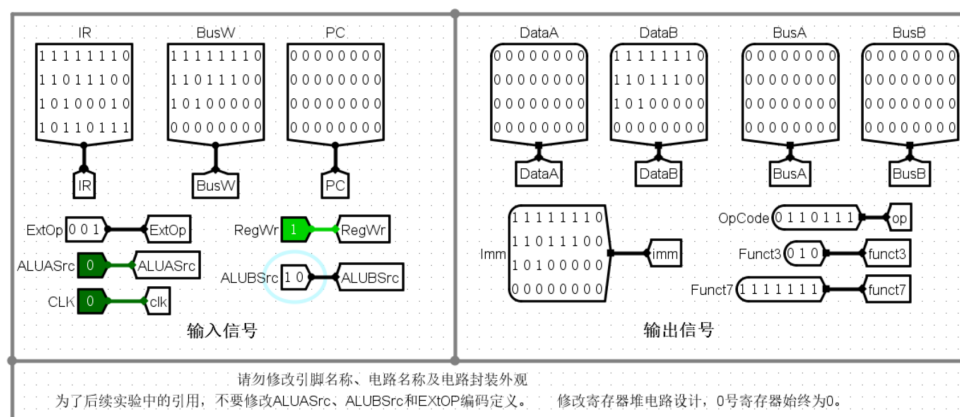
以下为取操作数部件输入输出数据列表。

PC	IR	BusW	控制信号值	指令功能 / 汇编语句	DataA、DataB 输出值
0x0	0xfedca2b7	0xfedca000	ExtOp=1 RegWr=1 ALUASrc=0 ALUBSrc=2	lui x5,0xfedca	0x0 / 0xfedca000
0x4	0xf9c28293	0xfedc9f9c	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=2	addi x5,x5,-100	0xfedca000 / 0xfffff9c
0x8	0x01006013	0x10	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=2	ori x0,x0,0x10	0x0 / 0x10
0xc	0x06502223	0x64	ExtOp=2 RegWr=0 ALUASrc=0 ALUBSrc=2	sw x5,0x64(x0)	0x0 / 0x64
0x10	0x06400303	0xfffff9c	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=2	lb x6,0x64(x0)	0x0 / 0x64
0x14	0x06601423	0x68	ExtOp=2 RegWr=0 ALUASrc=0 ALUBSrc=2	sh x6,0x68(x0)	0x0 / 0x68

PC	IR	BusW	控制信号值	指令功能 / 汇编语句	DataA、DataB 输出值
0x18	0x06405383	0x9f9c	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=2	lhu x7,0x64(x0)	0x0 / 0x64
0x1c	0x06701623	0x6c	ExtOp=2 RegWr=0 ALUASrc=0 ALUBSrc=2	sh x7,0x6c(x0)	0x0 / 0x6c
0x20	0x4042d413	0xffedc9f9	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=2	srai x8,x5,0x4	0xfedc9f9c / 0x404
0x24	0x006444b3	0x123665	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	xor x9,x8,x6	0xffedc9f9 / 0xfffff9c
0x28	0x00649533	0x50000000	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	sll x10,x9,x6	0x123665 / 0xfffff9c
0x2c	0x008505b3	0x4fedc9f9	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	add x11,x10,x8	0x50000000 / 0xffedc9f9
0x30	0x00b2a633	0x1	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	slt x12,x5,x11	0xfedc9f9c / 0x4fedc9f9
0x34	0x00b2b6b3	0x0	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	sltu x13,x5,x11	0xfedc9f9c / 0x4fedc9f9
0x38	0x40b287b3	0xaeed5a3	ExtOp=0 RegWr=1 ALUASrc=0 ALUBSrc=0	sub x15,x5,x11	0xfedc9f9c / 0x4fedc9f9
0x3c	0x06f02823	0x70	ExtOp=2 RegWr=0 ALUASrc=0 ALUBSrc=2	sw x15,0x70(x0)	0x0 / 0x70
0x40	0x0067c263	0x1	ExtOp=3 RegWr=0 ALUASrc=0 ALUBSrc=0	blt x15,x6,0x4	0xaeed5a3 / 0xfffff9c
0x44	0x0067d263	0x1	ExtOp=3 RegWr=0 ALUASrc=0 ALUBSrc=0	bge x15,x6,0x4	0xaeed5a3 / 0xfffff9c
0x48	0x0040086f	0x4c	ExtOp=4 RegWr=1 ALUASrc=1 ALUBSrc=1	jal x16,0x4	0x48 / 0x4

PC	IR	BusW	控制信号值	指令功能 / 汇编语句	DataA、DataB 输出值
0x4c	0x004808e7	0x50	ExtOp=0 RegWr=1 ALUASrc=1 ALUBSrc=1	jalr x17,x16,0x4	0x4c / 0x4
0x50	0x00001917	0x1050	ExtOp=1 RegWr=1 ALUASrc=1 ALUBSrc=2	auipc x18,0x1	0x50 / 0x1000

取操作数部件 IDU 仿真测试图



3.4.4 错误现象及分析

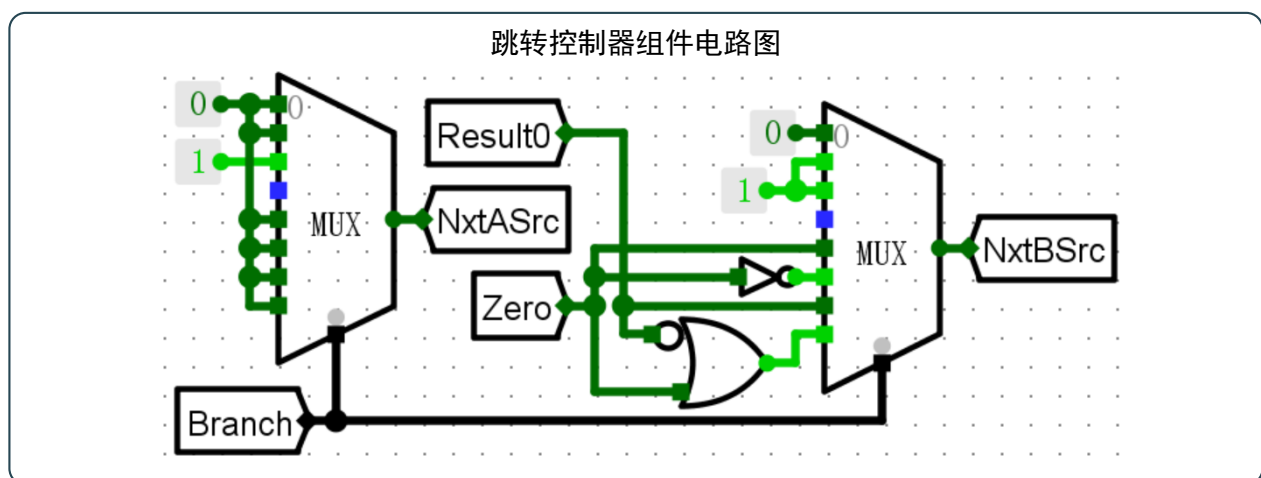
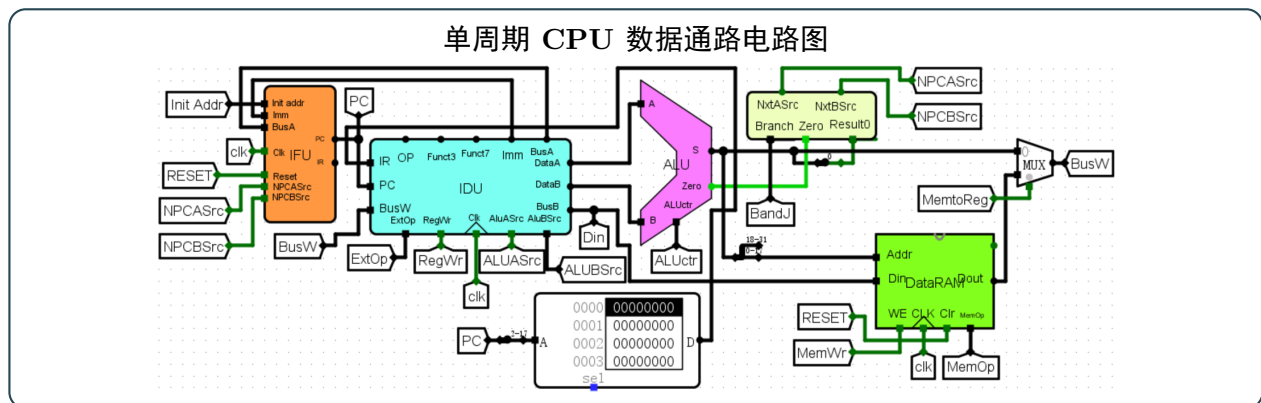
一开始的错误在于将两种立即数拓展方式的控制型号理解反了，使得前两组测试数据的输出与答案不符。在添加了一个非门后，问题得以解决。

3.5 数据通路实验

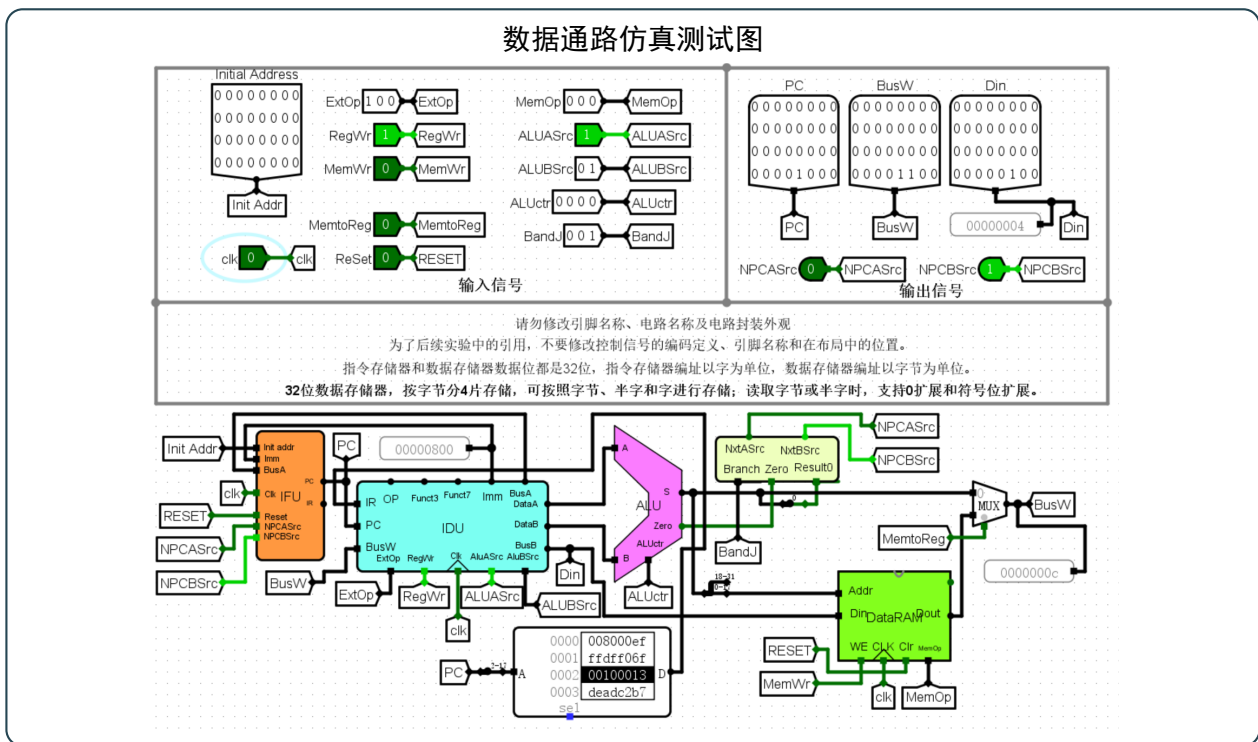
3.5.1 实验整体方案设计

- 顶层模块设计：本实验电路较为简单，不需要顶层模块设计图。
- 主要子电路：IFU、IDU、ALU、DataRAM、BandJ、寄存器堆等。

3.5.2 实验原理图和电路图



3.5.3 实验数据仿真测试图



序号	指令	控制信号赋值，未列出的控制信号值为 0。	输出信号值				
			PC	BusW	Din	NPCASrc	NPCBSrc
1	fedca2b7	ExtOp=1, RegWr=1, ALUBSrc=2, ALUctr=f	0x400	0xfedca000	0x0	0	0
2	f9c28293	RegWr=1, ALUBSrc=2	0x404	0xfedc9f9c	0x0	0	0
3	01006013	RegWr=1, ALUBSrc=2, ALUctr=6	0x408	0x10	0x0	0	0
4	06502223	ExtOp=2, MemWr=1, ALUBSrc=2	0x40c	0x64	0xfedc9f9c	0	0
5	06400303	MemOp=5, RegWr=1, ALUBSrc=2, MemtoReg=1	0x410	0xfffff9c	0x0	0	0
6	06601423	ExtOp=2, MemWr=1, MemOp=6, ALUBSrc=2	0x414	0x68	0xfffff9c	0	0
7	06405383	MemOp=2, RegWr=1, ALUBSrc=2, MemtoReg=1	0x418	0x9f9c	0x0	0	0
8	06701623	ExtOp=2, MemWr=1, MemOp=6, ALUBSrc=2	0x41c	0x6c	0x9f9c	0	0
9	4042d413	RegWr=1, ALUBSrc=2, ALUctr=d	0x420	0xffedc9f9	0x0	0	0
10	006444b3	RegWr=1, ALUctr=4	0x424	0x123665	0xfffff9c	0	0
11	00649533	RegWr=1, ALUctr=1	0x428	0x50000000	0xfffff9c	0	0
12	008505b3	RegWr=1	0x42c	0x4fedc9f9	0xffedc9f9	0	0
13	00b2a633	RegWr=1, ALUctr=2	0x430	0x1	0x4fedc9f9	0	0
14	00b2b6b3	RegWr=1, ALUctr=3	0x434	0x0	0x4fedc9f9	0	0
15	40b287b3	RegWr=1, ALUctr=8	0x438	0x0	0x4fedc9f9	0	0
16	06f02823	ExtOp=2, MemWr=1, ALUBSrc=2	0x43c	0x70	0x0	0	0
17	0067c263	ExtOp=3, ALUctr=2, BandJ=6	0x440	0x0	0xfffff9c	0	0
18	0067d263	ExtOp=3, ALUctr=2, BandJ=7	0x444	0x0	0xfffff9c	0	1
19	0040086f	ExtOp=4, RegWr=1, ALUABSrc=1, ALUBSrc=1, BandJ=1	0x448	0x44c	0x0	0	1
20	004808e7	RegWr=1, ALUABSrc=1, ALUBSrc=1, BandJ=2	0x44c	0x450	0x0	1	1

序号	指令	控制信号赋值, 未列出的控制信号值为 0。	输出信号值				
			PC	BusW	Din	NPCASrc	NPCBSrc
21	00001917	ExtOp=1, RegWr=1, ALUABSrc=1, ALUBSrc=2	0x450	0x1450	0x0	0	0

3.5.4 错误现象及分析

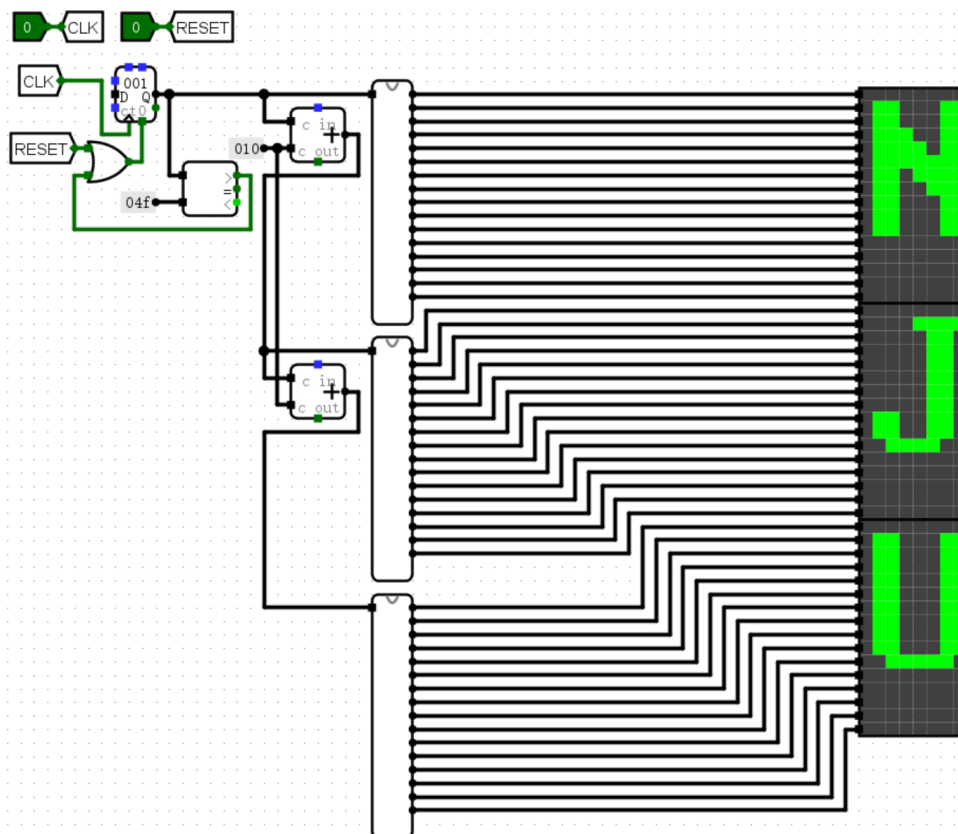
错误来源于加载了错误的指令数据。在加载正确的指令数据后, 正确通过了所有测试数据。

4 思考题

4.1 如何利用 ROM 实验实现滚动显示的功能, 在 3 个 LED 点阵矩阵中, 左右滚动显示 5 个 ASCII 字符?

可以在 ROM 中存储 5 个 ASCII 字符的点阵字模数据, 并设计一个计数器来控制当前显示的字符位置。每当计数器增加时, 显示的字符就向左滚动一个位置。当计数器达到一定值 (0x4f) 时, 重新回到初始位置, 实现循环滚动显示的效果。电路文件存储在 lab5.6.circ 中。

滚动显示布局图



4.2 分析说明如果 PC 寄存器、寄存器堆和数据存储器写入数据的时钟信号触发边沿不一致，则对程序执行有何影响？

如果 PC、寄存器堆和数据存储器的写入边沿不一致，就会破坏单周期 CPU 中“一个周期内状态同时更新”的假设，使各部件的写入与下一条指令的取数、译码不同步。这样容易产生读到旧值或未稳定数据的情况，造成数据冒险、写回错误、访存错误，严重时还会出现跳转地址错误或程序执行顺序紊乱。因此，通常应让这些时序部件尽量使用同一触发边沿，或通过严格的两相时钟设计保证时序正确。

4.3 在 CPU 启动执行后，如何实现在当前程序结束后，CPU 不再继续往下执行？

可以在程序末尾加入一条结束指令，例如 `ecall`，并在控制器中把它译码为停机信号，使 `PCWrite` 失效，从而冻结 PC，不再继续取后续指令；同时也可关闭寄存器堆和数据存储器的写使能，避免产生无意义的状态更新。若电路中没有专门的停机控制，也可以把最后一条指令写成跳转到自身的指令，例如 `jal x0, 0`，让 CPU 进入无限循环，从效果上停止继续向下执行。